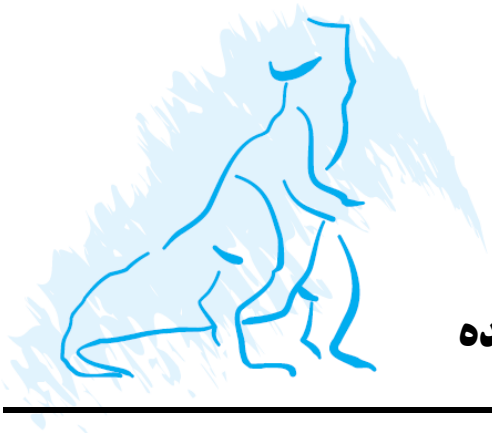


فصل ۵

زمانبندی پردازنده

زمانبندی پردازنده، اساس سیستم‌عامل‌های چندبرنامگی است. سیستم‌عامل با انتقال پردازنده مابین پردازه‌ها می‌تواند کامپیوتر را پربارتر نماید. در این فصل، مفاهیم اساسی زمانبندی پردازنده را معرفی کرده و چندین الگوریتم زمانبندی پردازنده، اعم از سیستم‌های بلادرنگ را نشان می‌دهیم. همچنین به مسأله‌ی انتخاب یک الگوریتم برای یک سیستم خاص نیز می‌پردازیم.

در فصل ۴، نخ‌ها را برای مدل پردازه معرفی کردیم. در سیستم‌عامل‌های مدرن، در حقیقت این نخ‌های سطح هسته—نه پردازه‌ها—هستند که توسط سیستم‌عامل زمانبندی می‌شوند. با این وجود، غالباً تعابیر «زمانبندی پردازه» و «زمانبندی نخ» به جای هم استفاده می‌شوند. در این فصل، زمانی‌که مفاهیم کلی زمانبندی را مد نظر داریم از *زمانبندی پردازه* و در صورتی‌که در رابطه با اصول خاص نخ‌ها صحبت می‌کنیم از *زمانبندی نخ* استفاده خواهیم کرد.

با تشابهی، ما در فصل ۱ بیان کردیم که چگونه یک *هسته* واحد محاسباتی مبنایی یک پردازنده است، و این که یک پردازه روی یک هسته‌ی پردازنده اجرا می‌شود. با این وجود، در بسیاری از نمونه‌های این فصل، موقعی که از اصطلاح کلی زمان‌بندی یک پردازه برای «اجرا بر روی یک پردازنده» استفاده می‌کنیم، تلویحاً منظور ما این است که پردازه بر روی یک هسته‌ی پردازنده اجرا می‌شود.

اهداف فصل

- بیان الگوریتم‌های مختلف زمان‌بندی پردازنده.
- ارزیابی الگوریتم‌های زمان‌بندی پردازنده بر اساس معیارهای زمان‌بندی.
- تشریح ملاحظات مربوط به زمان‌بندی چندپردازنده و چندهسته‌ای.
- بیان الگوریتم‌های مختلف زمان‌بندی بلادرنگ.
- بیان الگوریتم‌های زمان‌بندی استفاده شده در سیستم‌عامل‌های ویندوز، لینوکس، و سولاریس.
- به‌کارگیری مدل‌سازی و شبیه‌سازی برای ارزیابی الگوریتم‌های زمان‌بندی پردازنده.
- طراحی یک برنامه‌ای که چندین الگوریتم‌های متفاوت زمان‌بندی پردازنده را پیاده‌سازی می‌کند.

۵-۱ مفاهیم پایه

در یک سیستم با یک هسته‌ی پردازنده‌ی منفرد، فقط یک پردازه می‌تواند در هر لحظه اجرا شود. سایر پردازه‌ها بایستی منتظر بمانند تا این که هسته‌ی پردازنده رها شده و بتواند دوباره زمانبندی شود. هدف چندبرنامگی این است که در تمامی زمان‌ها پردازه‌ای برای اجرا داشته باشیم تا بهره‌وری پردازنده بیشینه گردد. این اصل به طور نسبی ساده است. هر پردازه مدت زمانی اجرا می‌شود تا این که به ناچار برای تکمیل درخواستی، نوعاً I/O منتظر می‌ماند. در یک سیستم کامپیوتر ساده، پردازنده فقط بیکار می‌نشیند. تمامی این زمان انتظار به هدر می‌رود؛ هیچ‌گونه کار مفیدی انجام نمی‌شود. با چندبرنامگی سعی داریم تا این زمان را پربارتر کنیم. چندین پردازه همزمان در حافظه نگه‌داری می‌شوند. هر موقع که پردازه‌ای به ناچار منتظر می‌ماند، سیستم‌عامل پردازنده را از آن گرفته و در اختیار پردازه‌ای دیگر قرار می‌دهد. این جریان ادامه می‌یابد. هر زمان که پردازه‌ای ناگزیر منتظر می‌شود، پردازه‌ای دیگر می‌تواند از پردازنده بهره ببرد. در یک سیستم چندهسته‌ای، این مفهوم مشغول نگه‌داشتن پردازنده به تمامی هسته‌های پردازشی روی سیستم تعمیم داده می‌شود.

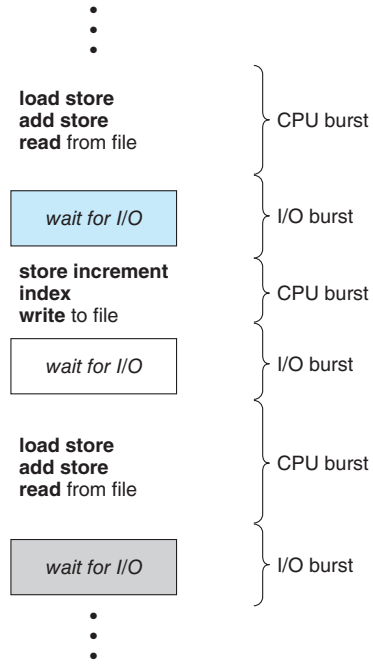
این نوع زمانبندی، مبنای کار سیستم‌عامل است. تقریباً تمامی منابع کامپیوتری پیش از این که مورد استفاده قرار بگیرند، زمانبندی می‌شوند. البته، پردازنده یکی از منابع اصلی کامپیوتر به حساب می‌آید. بنابراین، زمانبندی پردازنده در طراحی سیستم‌عامل نقش محوری دارد.

۵-۱-۱ دور انفجار پردازنده- I/O

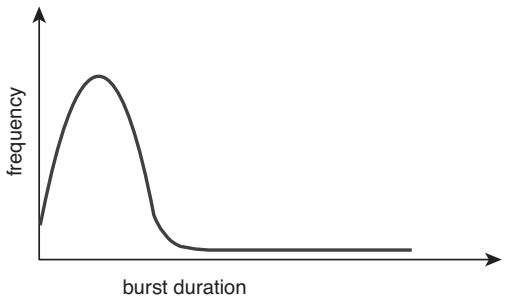
موفقیت زمانبندی پردازنده به ویژگی پردازه‌های مشاهده شده، بستگی دارد: اجرای پردازه شامل **دوری** از اجرا در پردازنده و انتظار I/O می‌باشد. پردازه‌ها با تناوبی میان این دو حالت قرار دارند. اجرای پردازه با یک **انفجار پردازنده**[1] آغاز می‌شود. به دنبال آن یک **انفجار I/O** صورت می‌گیرد که متعاقب آن هم انفجار دیگری از پردازنده اجرا می‌شود، سپس انفجار دیگر I/O و الی آخر. سرانجام، آخرین انفجار پردازنده با یک درخواست سیستمی مبنی بر خاتمه‌ی اجرا پایان می‌پذیرد (شکل ۵-۱).

فواصل انفجارهای پردازنده با قیاس بزرگی اندازه‌گیری شده‌اند. اگرچه این انفجارها از پردازه‌ای به پردازه‌ای دیگر و از کامپیوتری به کامپیوتر دیگر تفاوت زیادی دارند، منحنی فرکانس آنها شبیه شکلی است که در شکل ۵-۲ نشان داده می‌شود. منحنی به طور کلی به شکل نمایی یا فوق‌نمایی[2] با تعداد زیادی انفجار کوتاه پردازنده و تعداد کمی انفجار طولانی پردازنده است. برنامه‌ای که نوعاً عطش I/O دارد، تعداد انفجارهای کوتاه پردازنده‌اش زیاد است. در حالی‌که، برنامه‌ای که عطش پردازنده دارد، ممکن است انفجارهای پردازنده‌اش طولانی اما کم باشند. این توزیع می‌تواند در زمان پیاده‌سازی یک الگوریتم زمانبندی پردازنده اهمیت داشته باشد.

1. CPU burst
2. Hyperexponential



شکل ۵-۱ دنباله‌ی متناوب انفجارهای پردازنده و I/O.



شکل ۵-۲ هیستوگرام طول انفجارهای پردازنده.

۵-۱-۲ زمان‌بند پردازنده

هر موقع که پردازنده بیکار می‌شود، سیستم‌عامل بایستی یکی از پردازه‌های صف آماده را برای اجرا انتخاب کند. فرایند انتخاب توسط **زمان‌بند پردازنده** انجام می‌شود که یک پردازه را از پردازه‌های داخل حافظه انتخاب می‌کند که برای اجرا آماده است و پردازنده را به آن پردازه تخصیص می‌دهد.

توجه داشته باشید که صف آماده، الزاماً یک صف اولین-ورود، اولین-خروج (FIFO) نیست. همان‌گونه که در بررسی الگوریتم‌های زمانبندی مختلف خواهیم دید، صف آماده می‌تواند به صورت یک صف FIFO، الویتی، درختی یا به صورت ساده به شکل یک لیست پیوندی نامرتب

پیاده‌سازی شود. با این همه، این مفهوم وجود دارد که تمامی پردازه‌های صف آماده منتظر به خط ایستاده‌اند تا شانسی برای اجرا در پردازنده به‌دست آورند. رکوردهای صف به طور کلی PCBها[1] هستند.

۵-۱-۳ زمانبندی پس‌گرفتنی

تصمیم‌گیری‌های زمانبندی پردازنده می‌تواند تحت شرایط چهارگانه‌ی زیر صورت بگیرد:

۱- زمانی‌که پردازه‌ای از حالت اجرا به حالت انتظار منتقل می‌شود (برای مثال، به دنبال یک درخواست I/O یا احضار wait() برای انتظار ختم یک پردازه‌ی فرزند)
۲- زمانی‌که پردازه‌ای از حالت اجرا به حالت آماده منتقل می‌شود (برای مثال، موقعی که یک وقفه اتفاق می‌افتد)
۳- زمانی‌که پردازه‌ای از حالت انتظار به حالت آماده منتقل می‌شود (برای مثال، با تکمیل I/O)
۴- زمانی‌که پردازه‌ای خاتمه می‌پذیرد

برای شرایط ۱ و ۴، هیچ انتخابی با تعابیر زمانبندی در کار نیست. پردازه‌ی جدیدی (اگر در صف آماده باشد) می‌بایست برای اجرا انتخاب شود. با این وجود، برای شرایط ۲ و ۳، یک انتخاب امکان‌پذیر است.

زمانی‌که زمانبندی فقط در شرایط ۱ و ۴ صورت می‌گیرد، می‌گوییم که طرح زمانبندی **غیرقابل پس‌گیر**[2] یا **تعاونی**[3] است. در غیر این صورت، **پس‌گرفتنی**[4] است. تحت زمانبندی غیرقابل پس‌گیر، همین که پردازنده به پردازه تخصیص می‌یابد، پردازه پردازنده را نگه می‌دارد تا این که آن را با خاتمه و یا انتقال به حالت انتظار رها کند. تقریباً تمامی سیستم‌عامل‌های مدرن اعم از ویندوز، macOS، لینوکس، و یونیکس از الگوریتم‌های زمان‌بندی پس‌گرفتنی استفاده می‌کنند.

متأسفانه، زمانبندی پس‌گرفتنی موقعی که داده‌ها مابین چندین پردازه مشترک هستند، می‌تواند منجر به شرایط مسابقه شود. وضعیتی را در نظر بگیرید که دو پردازه داده‌هایی را به اشتراک می‌گذارند. در حالی‌که یک پردازه داده‌های مشترک را به روز می‌کند، پس‌گرفته می‌شود تا پردازه‌ی دوم بتواند اجرا شود. سپس، پردازه‌ی دوم سعی می‌کند تا داده‌هایی را بخواند که در یک وضعیت ناسازگار هستند. این موضوع با جزئیات بیشتر در فصل ۶ مورد بحث و بررسی قرار خواهد گرفت.

پس‌گیری بر طراحی هسته‌ی سیستم‌عامل نیز تأثیر می‌گذارد. طی پردازش یک فراخوانی سیستمی، این امکان وجود دارد که هسته مشغول کاری از جانب یک پردازه باشد. این اعمال ممکن است تغییرات داده‌های مهم هسته (برای نمونه، صف‌های I/O) را نیز شامل شوند. چه اتفاقی می‌افتد، در صورتی که پردازه‌ای در میان این تغییرات پس‌گرفته شود و هسته (یا درایور دستگاه) نیاز به خواندن یا تغییر همان ساختار داشته باشد؟ شرایط قابل پیش‌بینی نیست. همان‌گونه که در

1. Process control blocks
2. Nonpreemptive
3. Cooperative
4. Preemptive

بخش ۶-۲ بحث خواهد شد، هسته‌ی سیستم‌عامل‌ها می‌تواند به‌صورت غیرقابل پس‌گیر و یا پس‌گیر طراحی شود. یک هسته‌ی غیرقابل پس‌گیر منتظر یک فراخوانی سیستمی خواهد ماند تا تکمیل شود یا منتظر یک پردازه خواهد ماند تا مسدود شود در حالی که به‌انتظار تکمیل I/O می‌باشد، قبل از این که یک برگردان متن انجام دهد. این طرح اطمینان می‌دهد که ساختار هسته ساده بماند، چرا که هسته پردازه‌ای را پس نخواهد گرفت در حالی‌که ساختمان‌داده‌های هسته در یک حالت ناسازگار باشند. متأسفانه، این مدل اجرایی هسته در پشتیبانی محاسبات بلادرنگ که وظیفه‌ها می‌بایست اجرای خود را در یک قاب زمانی مشخص به پایان برسانند، ضعیف عمل می‌کند. در بخش ۵-۶، نیازمندی‌های زمانبندی سیستم‌های بلادرنگ را بحث و بررسی می‌کنیم. یک هسته‌ی پس‌گیر راهکارهایی همچون قفل‌های انحصار متقابل[1] را لازم دارد تا موقع دسترسی ساختمان‌داده‌های مشترک هسته از شرایط رقابت[2] پیشگیری کند. اکنون، اکثر سیستم‌عامل‌های مدرن، موقعی که در مد هسته اجرا می‌شوند کاملاً پس‌گرفتنی هستند.

چون طبق تعریف، وقفه‌ها در هر لحظه می‌توانند اتفاق بیفتند و چون هسته برای همیشه نمی‌تواند آنها را نادیده بگیرد، بایستی بخش‌هایی از کد که از وقوع وقفه‌ها متأثر می‌شوند در مقابل استفاده‌ی همزمان محافظت شوند. سیستم‌عامل بایستی در تمامی زمان‌ها وقفه‌ها را بپذیرد. در غیر این صورت، امکان دارد ورودی از دست برود یا خروجی رونویسی گردد. لذا، نباید این بخش‌های کد توسط چندین پردازه به طور همروند دستیابی شوند، وقفه‌ها با ورود به این بخش‌ها غیرفعال و دومرتبه با خروج از آنها فعال می‌شوند. توجه به این نکته اهمیت دارد که بخش‌های کد مربوط به غیرفعال‌کردن وقفه‌ها خیلی کم پیش می‌آیند و نوعاً چند دستور هم بیشتر نخواهند بود.

۵-۱-۴ اعزام‌کننده

مؤلفه‌ی دیگری که در کار زمانبندی پردازنده نقش دارد، **اعزام‌کننده**[3] است. اعزام‌کننده نهادی است که کنترل هسته‌ی پردازنده را در اختیار پردازه‌ای می‌گذارد که توسط زمانبند پردازنده انتخاب می‌شود. تحقق این هدف مشمول انجام اعمال زیر است:

- برگرداندن متن از یک پردازه به دیگری
- انتقال به مد کاربر
- پرش به مکان مناسبی در برنامه‌ی کاربر برای از سرگیری آن برنامه

اعزام‌کننده بایستی تا حد امکان سریع عمل کند، چرا که طی هر برگردان متن احضار می‌شود. زمانی‌که سپری می‌شود تا اعزام‌کننده اجرای یک پردازه را متوقف و اجرای پردازه‌ای دیگر را شروع نماید به **دیرکرد اعزام**[4] معروف است و در شکل ۵-۳ نمایش داده می‌شود.

یک پرسش جالبی که بایستی در نظر گرفت، این است که برگردان‌های متن چند به وقت اتفاق می‌افتند؟ در سطح سیستم، تعداد برگردان‌های متن می‌تواند با استفاده از فرمان vmstat

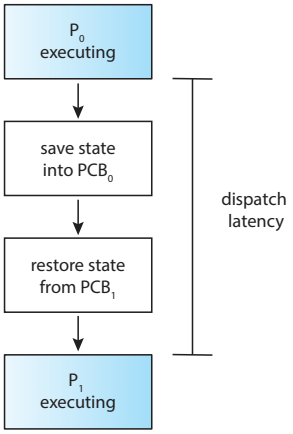
1. Mutex locks
2. Race conditions
3. Dispatcher
4. Dispatch latency

بدست آید که در سیستم‌های لینوکس وجود دارد. در زیر خروجی فرمان (که قیچی شده است) نمایش داده می‌شود:

```
vmstat 1 3
```

این فرمان ۳ خط خروجی با تأخیر ۱ ثانیه‌ای فراهم می‌کند:

```
------cpu-----
24
225
339
```



شکل ۵-۳ نقش اعزام‌کننده.

خط اول تعداد متوسط برگردان‌های متن را در ۱ ثانیه از زمان بوت شدن سیستم نشان می‌دهد، و دو خط بعدی تعداد برگردان‌های متن را در دو فاصله‌ی ۱ ثانیه‌ای نمایش می‌دهد. از زمانی که این ماشین بوت شده است، به‌طور متوسط ۲۴ برگردان متن در هر ثانیه اتفاق افتاده است. و در ثانیه گذشته، ۲۲۵ برگردان متن با ۳۳۹ برگردان متن در ثانیه‌ی قبل از آن انجام گرفته است.

می‌توانیم برای تعیین تعداد برگردان متن‌های یک پردازه‌ی مشخص از سیستم‌فایل proc/ نیز استفاده می‌کنیم. برای مثال، محتوای فایل proc/2166/status/ آمار مختلف پردازه‌ای با pid = 2166 را لیست خواهد نمود. فرمان

```
cat /proc/2166/status
```

خروجی زیر را فراهم می‌کند:

```
voluntary_ctxt_switches        150
nonvoluntary_ctxt_switches     8
```

این خروجی تعداد برگردان‌های متن را در طی حیات پردازه نشان می‌دهد. به تمایز بین برگردان‌های متن *داوطلبانه* و *غیرداوطلبانه* توجه کنید. برگردان متن داوطلبانه موقعی اتفاق می‌افتد که پردازه کنترل پردازنده را پس می‌دهد چون منبعی لازم دارد که در حال حاضر در

دسترس نیست (همچون انسداد برای I/O). برگردان متن غیرداوطلبانه موقعی اتفاق می‌افتد که پردازنده از یک پردازه پس گرفته می‌شود، مثلاً موقعی که سهم زمانی آن تمام شده یا توسط یک پردازه‌ای با الویت بالاتر پس‌گرفته می‌شود.

۵-۲ معیارهای زمانبندی

الگوریتم‌های زمانبندی متفاوت پردازنده، خواص متفاوتی دارند و انتخاب یک الگوریتم خاص می‌تواند یک رده از پردازه‌ها را نسبت به سایر پردازه‌ها برتری ببخشد. در این که، کدام الگوریتم را برای یک شرایط خاص انتخاب کنیم، بایستی خواص الگوریتم‌های مختلف را مد نظر قرار بدهیم.

معیارهای زیادی برای مقایسه‌ی الگوریتم‌های زمانبندی پردازنده مطرح شده‌اند. هر یک از مشخصه‌ها که برای مقایسه استفاده شود، می‌تواند در رابطه با قضاوت ما در مورد بهترین الگوریتم، تفاوت قابل‌توجهی را موجب شود. این معیارها عبارتند از:

- **بهره‌وری پردازنده**: می‌خواهیم تا جایی که ممکن است پردازنده را مشغول نگه بداریم. تصور ما این است که بهره‌وری پردازنده می‌تواند از ۰ تا ۱۰۰ درصد تغییر نماید. این میزان در یک سیستم واقعی، بایستی از ۴۰ درصد (برای سیستم سبک‌بار) تا ۹۰ درصد (برای سیستم سنگین‌بار) در تغییر باشد. (بهره‌وری پردازنده در سیستم‌های لینوکس، macOS، و یونیکس با استفاده از فرمان top بدست آید.)
- **بازده**: اگر پردازنده به اجرای پردازه‌ها مشغول باشد، در این صورت کار انجام می‌شود. یک میزان از کار، تعداد پردازه‌هایی است که در واحد زمان تکمیل می‌شوند که **بازده** نام دارد. این نرخ در مورد پردازه‌های طولانی می‌تواند به یک پردازه در چند ثانیه هم برسد؛ برای تراکنش‌های کوتاه، این امکان هست که بازده به ده‌ها پردازه در ثانیه نیز برسد.
- **زمان گردش**[1]: مهمترین معیار از دید پردازه‌ای خاص، مدت زمانی است که سپری می‌شود تا آن پردازه اجرا شود. فاصله‌ی زمانی میان ارسال یک پردازه تا زمان تکمیل آن، زمان گردش است. زمان گردش برابر مجموع دوره‌های زمانی است که پردازه در انتظار در صف آماده، اجرا در پردازنده و انجام I/O سپری می‌نماید.
- **زمان انتظار**: الگوریتم زمانبندی پردازنده بر میزان زمانی که پردازه اجرا می‌شود یا I/O انجام می‌دهد تأثیر نمی‌گذارد. آن فقط روی مدت زمانی که پردازه در صف آماده برای انتظار سپری خواهد کرد تأثیر دارد. *زمان انتظار*، برابر مجموع دوره‌های زمانی است که پردازه در صف آماده منتظر می‌ماند.
- **زمان پاسخ**: در سیستم محاوره‌ای ممکن است که زمان گردش به عنوان بهترین معیار محسوب نشود. در اغلب موارد، یک پردازه می‌تواند خیلی زود، مقداری خروجی تولید نماید و در حالی‌که نتایج قبلی برای کاربر نمایش داده می‌شوند به محاسبات نتایج جدید ادامه بدهد. بنابراین، معیار دیگر، مدت زمانی است که از

1. Turnaround time

ارسال یک درخواست تا تولید اولین پاسخ سپری می‌شود. این معیار، زمان پاسخ نامیده می‌شود که زمان سپری شده‌ای است تا اولین پاسخ شروع شود، نه زمانی که برای نمایش پاسخ گرفته می‌شود.

مطلوب‌تر است که بهره‌وری پردازنده و بازده بیشینه شود و زمان گردش، انتظار و پاسخ به حداقل میزان خود برسد. در بیشتر موارد، اندازه‌ی متوسط را بهینه می‌کنیم. با این وجود، در بعضی شرایط علاقه داریم که به جای میانگین، مقادیر حداقل یا حداکثر بهینه شوند. برای مثال، برای این که تضمین کنیم تمامی کاربران سرویس خوبی می‌گیرند، ممکن است بخواهیم حداکثر زمان پاسخ را کمینه کنیم.

تحلیل‌گران پیشنهاد می‌دهند که در سیستم‌های محاوره‌ای (همچون یک PC رومیزی یا یک سیستم لپ‌تاپ)، مهم‌تر است که به جای این که میانگین زمان پاسخ را کمینه کنیم، واریانس زمان پاسخ را به حداقل برسانیم. سیستمی که از یک زمان پاسخ مقبول و قابل پیش‌بینی برخوردار است، می‌تواند مطلوب‌تر از سیستمی باشد که در حالت متوسط سریع‌تر عمل می‌کند، اما تغییرات زیادی دارد. با این وجود، در الگوریتم‌های زمانبندی پردازنده که واریانس را کمینه می‌سازند کار کمتری انجام شده است.

همچنان که الگوریتم‌های زمانبندی مختلف پردازنده را در بخش بعدی بحث می‌کنیم عملیات آنها را نیز نشان خواهیم داد. در یک نمایش واقعی، بایستی پردازه‌های زیادی لحاظ شوند که هر یک دنباله‌ای از صدها انفجار پردازنده و انفجار I/O را دارند. هرچند که برای سادگی مثال‌های خود، تنها یک انفجار پردازنده از پردازه‌ها را (به میلی‌ثانیه) مد نظر قرار خواهیم داد. معیار مقایسه‌ی ما، میانگین زمان انتظار است. راهکارهای ارزیابی دقیق‌تر در بخش ۵-۸ مورد بحث قرار می‌گیرند.

۵-۳ الگوریتم‌های زمانبندی

زمانبندی پردازنده با مسأله‌ی تصمیم‌گیری این که، هسته‌ی پردازنده را در اختیار کدام یک از پردازه‌های صف آماده قرار بدهد، سر و کار دارد. الگوریتم‌های زمانبندی متفاوت پردازنده فراوان هستند. در این بخش، چند مورد از آنها را بیان می‌کنیم. هرچند که اکثر معماری‌های مدرن پردازنده دارای هسته‌های پردازشی متعددی هستند، ما این الگوریتم‌های زمان‌بندی را در متن فقط یک هسته‌ی پردازشی در دسترس بیان می‌کنیم. یعنی، یک پردازنده‌ی منفردی که دارای یک هسته‌ی پردازشی منفرد است، لذا سیستم، قابلیت اجرای یک پردازه را در یک لحظه دارا می‌باشد.

۵-۳-۱ زمانبندی اولین-ورود، اولین-سرویس

تاکنون، ساده‌ترین الگوریتم زمانبندی پردازنده، الگوریتم زمانبندی **اولین-ورود، اولین-سرویس**[1] (**FCFS**) بوده است. در این طرح، پردازه‌ای پردازنده را برای نوبت اول به‌دست می‌گیرد که اول از همه درخواست داده باشد. پیاده‌سازی سیاست FCFS به راحتی با یک صف FIFO انجام

1. First-come, first-served scheduling algorithm